A Proposed Agentic RAG Framework for PostgreSQL Optimization in Scientific Computing Infrastructure

Jingni Huang, Department of Computer Science, University of Oxford

jingni.huang@kellogg.ox.ac.uk,

jingnih@gmail.com,

https://github.com/JNH2

Abstract

Scalable AI-Infra for PB-scale Metadata.

High- energy physics experiments generate massive datasets. Detectors digitize particle signals - light, charge, radiation – into structured data streams. Processing this correctly requires knowing the exact conditions at collection time: collider settings, detector alignment, magnetic fields, electronics parameters, ambient temperature and humidity.

However, conditions are never static. Detectors age, get recalibrated, reconfigured. So every configuration must carry an Interval of Validity (IOV), defining precisely when it is applied. Groups of IOVs form Global Tags - coherent, versioned snapshots of the full experimental setup, whether for real data or simulation.

The nopayloaddb system addresses this need as the HSF reference implementation, produced through collaborative expert work at CERN. Architecturally, it separates payload storage from metadata: conditions files live on a filesystem while the database tracks only their URLs and associated IOV - a lightweight, Kubernetes-native solution that adapts readily across experimental environments.

The system has demonstrated production maturity: it currently serves SPHENIX at Brookhaven, Belle II is in active migration, and Einstein Telescope has adopted it in its baseline computing model.

This work presents an Agentic RAG framework designed to extend existing infrastructure, simplify database observability workflow, and introduce AI-assisted PostgreSQL optimization.

The project implementation will be based on an Agentic RAG framework, used to autonomously diagnose and optimize PostgreSQL retrieval performance without modifying the database schema. By leveraging EXPLAIN plan analysis and Kubernetes' native sandbox mechanism, the system ensures the availability, stability, and scalability of petabyte-scale metadata in

high-concurrency environments. To ensure absolute stability in the production environment, we have introduced a "Human in a Loop" workflow: all AI-generated optimization suggestions must be reviewed by the administrator and then simulated and verified in a pre-release environment. Deployment to the production environment will only occur after performance improvements meet the expected targets; otherwise, a rollback mechanism will be triggered immediately.

1. Technology Stack and Tools

To ensure the scalability and reliability of the AI-assisted framework, an AI Optimization Microservice using Python will be implemented, structured with a clean architecture(Repository-Service-Controller pattern):

1.1Infrastructure and Orchestration

Deploy a StatefulSet PV cluster with Master and Replicas.

Kubernetes(miniKube): The primary container orchestration platform for hosting distributed PostgreSQL clusters and AI analytics microservices.

Helm/YAML: Used for package management to ensure reproducible deployment of the database, read-only replicas, and monitoring stack across different environments (pre-release and production).

Staging Environment: A dedicated sandbox environment cloned via Helm for securely validating AI-generated tuning recommendations before production deployment.

1.2 Applications and Data Entry Points

Django & Nginx: The core framework and web server for the nopayloaddb project. Nginx acts as a load balancer, while Django provides a RESTful API for metadata management.

PostgreSQL: A target relational database. This project focuses on optimizing its process-based architecture and managing resource contention.

1.3 Database Observability

pg_stat_statements: A key PostgreSQL extension for tracking execution statistics of all SQL statements, identifying high-latency "slow queries" for AI agent analysis.

EXPLAIN / ANALYZE: Database commands for generating detailed execution plans. This project specifically parses JSON output to understand the query planner's cost-based decisions.

Prometheus & Grafana: A combination of monitoring metrics for collecting and visualizing system-level indicators (CPU, RAM, disk I/O). These metrics provide contextual information for Root Cause Analysis (RCA).

1.4 AI and Intelligent Engine

LangChain: The core framework for orchestrating LLM workflows for reasoning. It manages the logic between the database observer, knowledge base, and inference agent.

RAG (Retrieval Augmentation Generation): The core AI strategy. Instead of relying on general LLM knowledge, the system retrieves specific authoritative tuning strategies from official documentation.

Vector Database (ChromaDB): Stores an embedded PostgreSQL optimization manual, supporting high-performance semantic search to find relevant tuning heuristics.

Chunking (Recursive Character Segmentation): A preprocessing technique that breaks down complex technical manuals into manageable fragments, ensuring LLM receives highly relevant and focused contextual information.

AI Agent: An autonomous inference unit that analyzes the EXPLAIN plan, evaluates system metrics, and determines the best remediation strategy (e.g., query rewriting or dynamic parameter tuning).

1.5 SQL Optimization Strategies Examples From Slow Queries to Efficient Commands

Examples for rewriting SQL without modifying the schema.

Redundant Join Elimination: ORMs often generate complex JOINs involving 5-10 tables, but some tables are only used to retrieve a field that isn't actually needed. AI willsuggest modifying the Django QuerySet to retain only the core joins, downgrading queries with O(N^k) complexity.

Subquery to JOIN Conversion:WHERE id IN (SELECT ...) can cause a full table scan with large datasets. AI suggests rewriting it as an INNER JOIN, utilizing PostgreSQL's Hash Join algorithm.

Covering Indexes:Queries require access to the table's raw data (Heap). AI recommends rewriting the SQL to only query fields already in the index (Index Only Scan), completely skipping the reading of data from the physical disk.

1.6 The Optimization Ledger

To ensure the traceability of AI interventions, the framework proposes an Optimization Ledger. For every identified bottleneck using EXPLAIN (ANALYZE, BUFFERS), the AI Agent will log the Pre- and Post-optimization execution plans. This ‘Internal Notebook’ serves two purposes:

Knowledge Reuse: Rapidly applying verified fixes to recurring query patterns under Hz-level concurrency.

Performance Regression Guard: Ensuring that any SQL refactoring or parameter tuning (e.g., shared_buffers) yields a measurable reduction in Disk I/O latency before being promoted to production

2. System Requirements

2.1Non-functional Requirements: Reliability, Scalability, and Maintainability

The framework ensures reliability through a "safety first" cycle: AI recommendations are validated in a pre-release sandbox before deployment to production and are supported by a rollback controller to recover from any regression errors. It achieves scalability through two layers: (1) an infrastructure layer where Django pods can scale horizontally in Kubernetes/miniKube to handle the increasing number of concurrent analytics requests; and (2) scalability may also be achieved at the RAG level (requires further investigation). Finally, the framework also considers maintainability, technically attempting to use explainable AI and building upon industry-standard tools (Helm, K8s, LangChain) to ensure the HSF community can easily audit and scale processes.

2.2 Functional Requirements

To address performance bottlenecks in the nopayloaddb system, this framework will provide the following key functionalities:

Automated Querying:Real-time querying of high-latency SQL queries using pg_stat_statements; Automatic generation of detailed execution plans in JSON format via EXPLAIN (ANALYZE, BUFFERS).

Context-Aware Knowledge Retrieval: A RAG-based engine capable of autonomously searching PostgreSQL documentation and the optimization knowledge base (ChromaDB) for solution discovery and detection.

AI-Driven Bottleneck Analysis: The AI Agent/LLM identifies root causes (e.g., memory overflow or insufficient work_mem) by analyzing obstacles and execution times encountered in the execution plan.

Prescriptive Fixes: Generation of executable optimization strategies, including: SQL Refactoring; Rewriting inefficient joins or subqueries; Parameter Tuning: Recommending dynamic adjustments to work_mem and shared_buffers.

Closed-Loop Validation System: A pre-release sandbox workflow that automatically applies recommendations in a secure environment, providing before-and-after performance reports for manual review and a rollback security mechanism.

3. Architecture and Workflow Overview

3.1 Microservice High-Level Architect in Minikube

The three parts are distributed across two environments, separated by namespaces: Production and Staging (Pre-release/Sandbox).

Environment 1A: Production Namespace

Pods: Nginx -> Django (Nopayloaddb) -> PgBouncer -> PostgreSQL (Master/Replica).

Core Logic: This is the data source. pg_stat_statements records raw slow query data here. The Nopayloaddb middleware stores suggestions for slow SQL modifications.

Environment 1B: Monitoring & AI Engine (Control Tower)

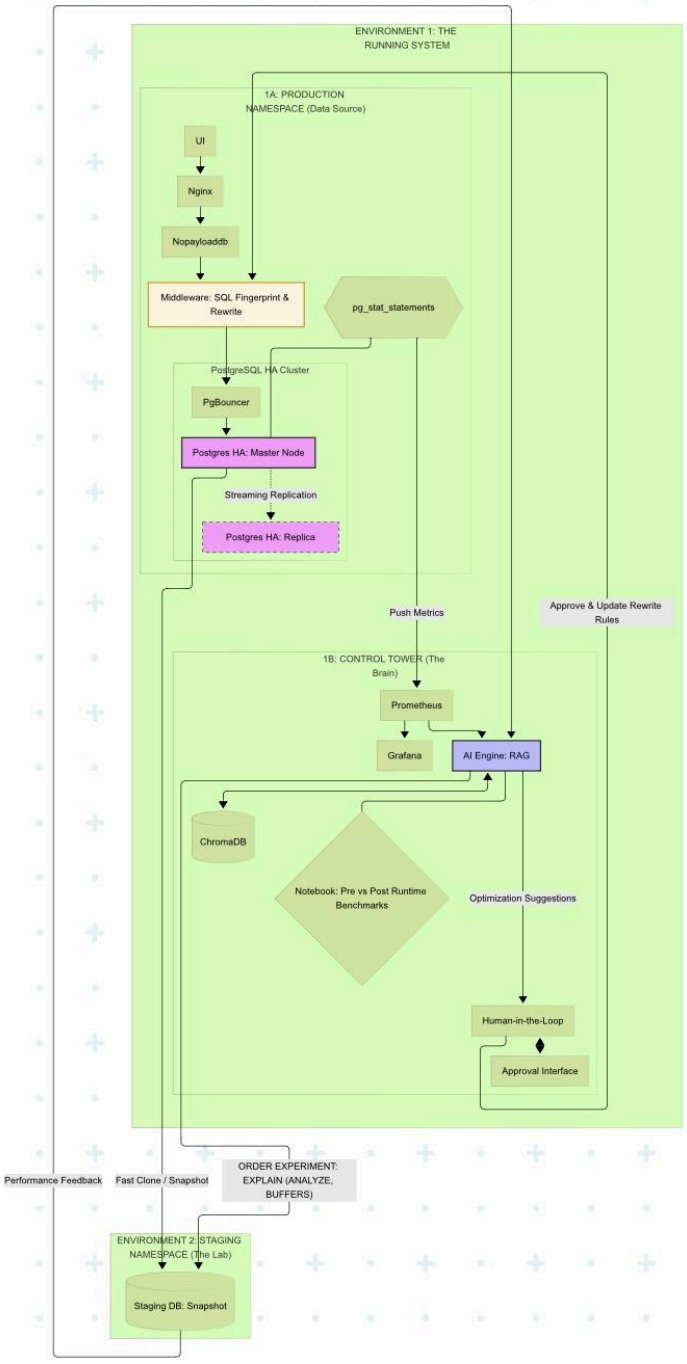
Pods: Prometheus, Grafana, AI Engine (RAG), Human-in-the-Loop

Core Logic: This is the brain. It pulls data from Environment 1A, detects slowdowns, and orders Environment 2 to begin experiments.

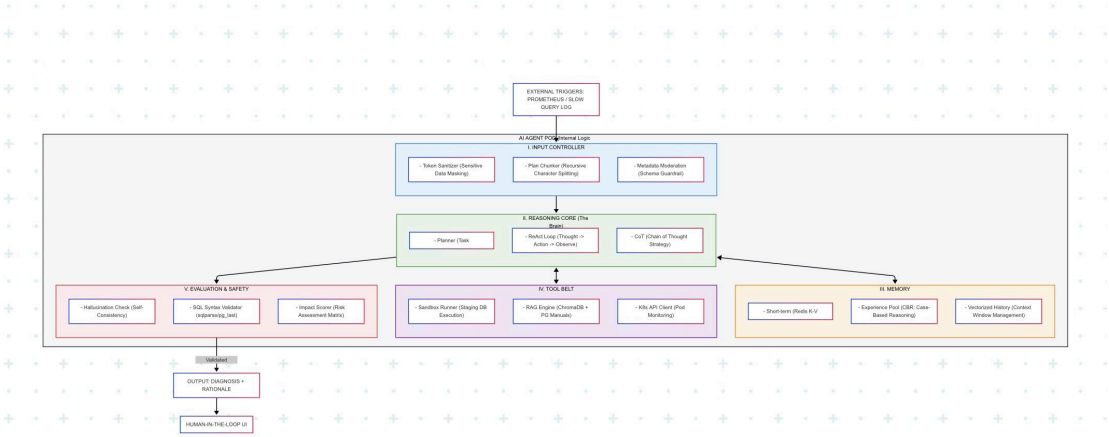
Environment 2: Staging/Sandbox Namespace

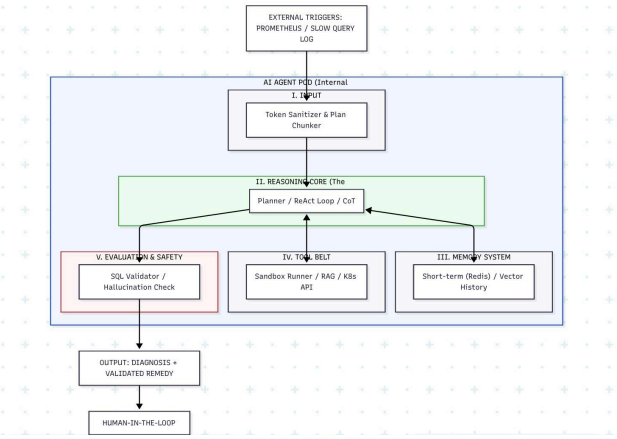
Pods: Staging DB (with snapshots of production data).

Core Logic: This is the operating room. The AI runs EXPLAIN (ANALYZE, BUFFERS) here. If the optimization is verified to be effective, it is then fed back to "Environment 1" for deployment through a human-in-the-loop process.

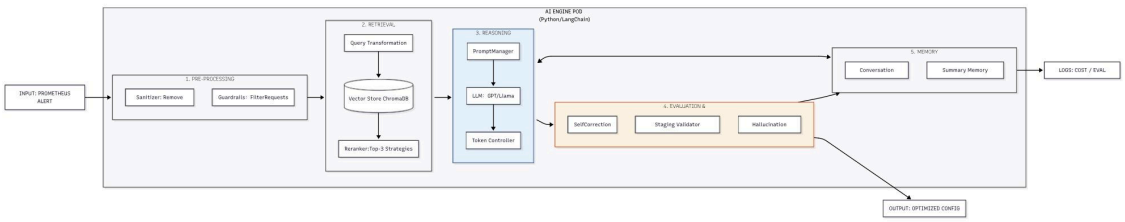

ENVIRONMENT 1: THE RUNNING SYSTEM
1A: PRODUCTION NAMESPACE (Data Source)
UI
Nginx
Nopayloaddb
Middleware: SQL Fingerprint & Rewrite
pg_stat_statements
PostgreSQL HA Cluster
PgBouncer
Postgres HA: Master Node
Streaming Replication
Postgres HA: Replica
Push Metrics
Approve & Update Rewrite Rules
1B: CONTROL TOWER (The Brain)
Prometheus
Grafana
AI Engine: RAG
ChromaDB
Notebook: Pre vs Post Runtime Benchmarks
Optimization Suggestions
Human-in-the-Loop
Approval Interface
Performance Feedback
Fast Clone / Snapshot
ORDER EXPERIMENT: EXPLAIN (ANALYZE, BUFFERS)
ENVIRONMENT 2: STAGING NAMESPACE (The Lab)
Staging DB: Snapshot

3.2 AI Agent Low-Level Architects



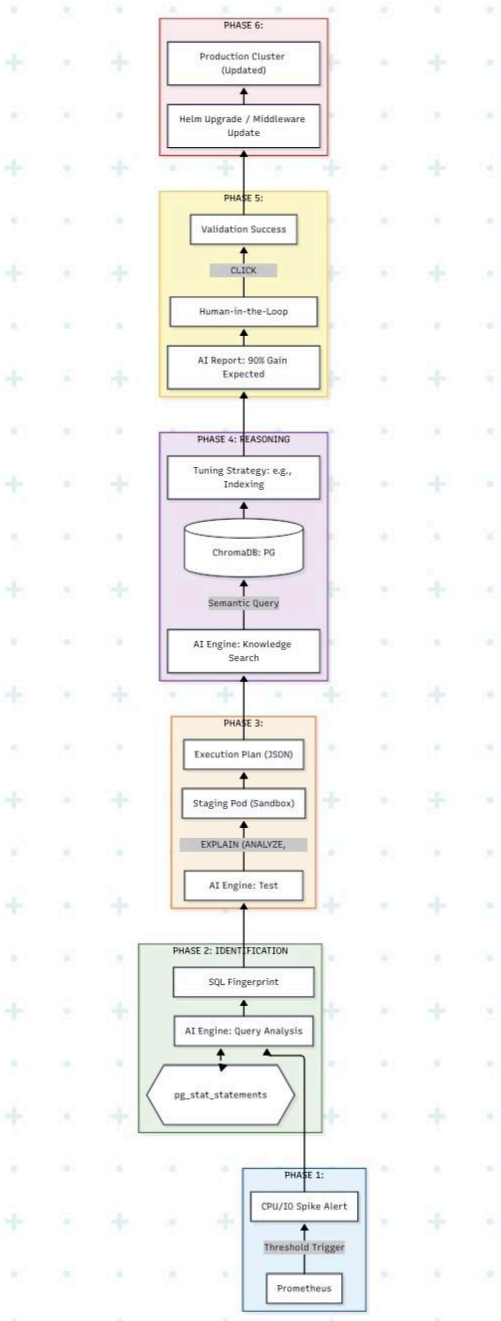


3.3 LLM-OPS ARCHITECTURE: THE AI DIAGNOSTIC AGENT INTERNALS



3.4 AI Diagnostic Path and Architecture

We divide the entire process into "Normal Business Flow" and "AI Diagnostic Flow".



Phase 1: Normal Business Flow (The Happy Path)

1. User inputs in the UI -> Nginx forwards -> Django generates SQL.

2. SQL passes through PgBouncer -> sent to the Replica Pod for execution.

3. During PostgreSQL execution, it first checks Shared Buffers, then the OS Cache, and finally reads the disk (PV).

4. pg_stat_statements silently records the execution time and I/O of this SQL statement in the background.

Phase 2: AI Diagnostic and Optimization Flow (The Optimization Loop)

1. Discovery: Prometheus detects excessive I/O wait on Replica 01 and triggers an alert to the AI Engine.

2. Location: The AI Engine accesses Replica 01's pg_stat_statements and captures the slowest SQL statement.

3. Experiment: The AI Engine runs the SQL in the Staging Pod with EXPLAIN (ANALYZE, BUFFERS).

    Analyze runs the SQL once to see real-time performance.

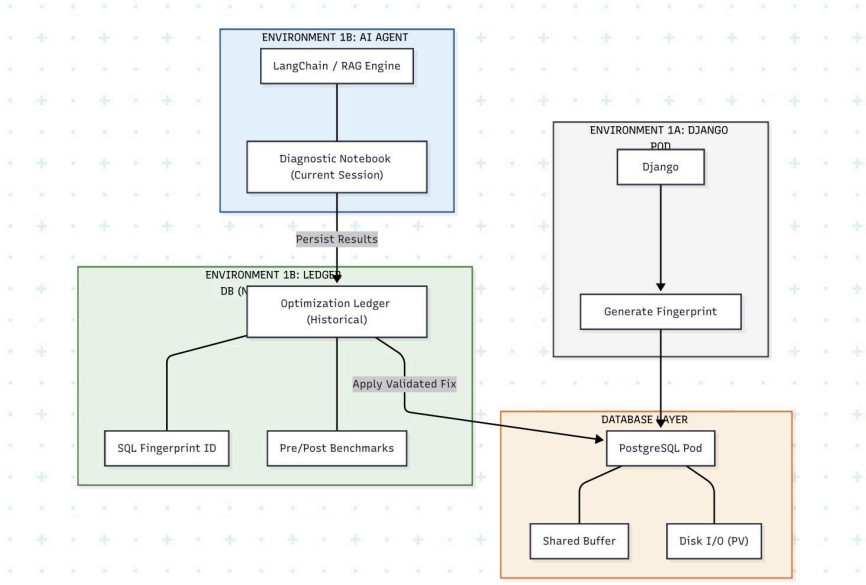
    Buffers checks how much data was read from slow disks.

4. Inference: The AI sends the execution plan to RAG. The RAG manual suggests: "Add an index or increase work_mem".

5. Approval (Human-in-the-loop): The AI displays a "Suggestion Report" in the UI, which the database administrator reviews and approves.

6. Deployment: The AI updates its configuration via Helm. After verifying the performance improvement in Staging, it is finally applied to Production.

3.5. The Optimization Ledger Workflows



4. Project Deliverables

This section concludes the work, clarifying the core and advanced development goals.
Core deliverables:
Infrastructure Deployment: A validated Helm Chart for the full stack (Nginx + Nopayloaddb + PgBouncer + PostgreSQL HA)
HA StatefulSet Backend: Implementation of a high-availibility Master-Replica PostgreSQL backend.
Diagnostic RAG Engine: A functional AI agent indexed with PostgreSQL.
Target Metric: Achieve > 90% syntax accuracy on a synthetic test set consisting of common performance anti-patterns.

Advanced Deliverables:

Automated Sandbox Validation: Risk-free testing of AI suggestions in a cloned environment using Volume Snapshots.

Closed-Loop Monitoring: Real-time tracking of latency changes post-deployment.

Manual Cross-Referencing: Automated linking of all AI recommendations to specific official documentation sections.

5. Risk Mitigation and Flexibility

Mitigate Strategies (Minimum Success Criteria)

Complex EXPLAIN Parsing: If JSON metadata extraction is too complex, fall back to standard text-based pattern matching.

LLM Illusion: Prefer RAG-based references (direct citations of the manual) over generative suggestions to ensure accuracy. Kubernetes Connectivity Issues

If the OpenShift environment is limited, ensure the entire process runs perfectly correctly in the Codespace Minikube environment as a baseline.

6. Future Work: Systematic Evaluation & Benchmarking

Upon the successful deployment of the core infrastructure, potential future expansions will focus on establishing a formal Evaluation Framework. This involves implementing quantitative metrics - such as Faithfulness and Answer Relevance - to benchmark the AI’s suggestions against official PostgreSQL standards. Additionally, an automated Safety Gate (LLM-as-a-Judge) could be integrated to provide a secondary layer of architectural audit and cost-verification before any

recommendation reaches the DBA, ensuring long-term system reliability and optimization precision.

7. Ethical, Legal, and Professional Issues

7.1 Ethical and Professional Impact

Accountability: To reduce the "AI illusion," the system enforces human-computer interaction verification, positioning AI as an auxiliary "co-pilot," with final decision-making power remaining with the Database Administrator (DBA).

Objectivity: All recommendations are based on the official PostgreSQL documentation (RAG) to minimize potential algorithmic bias.

7.2 Social and Workplace Dynamics

Enhancement: This tool focuses on automating tedious EXPLAIN analysis to assist, rather than replace, human expertise, thereby maintaining team trust.

Education: By providing transparent and referable reasoning, this tool serves as a technical resource for junior developers to learn the inner workings of complex databases.

7.3 Legal and Regulatory Compliance

CERN Policy: Any future integration with production logs will strictly adhere to CERN's internal data protection protocols.

Licenses: All open-source components (LangChain, ChromaDB, PG) and documentation will be used in full compliance with their respective licenses.

7.4 Reasons for Not Conducting Ethical Review at This Time

This project currently only involves technical framework design and anonymized metadata analysis. Since it does not involve human participants, the collection of sensitive personal data, or biological samples, a formal ethical review is not required at this stage. If real-time user telemetry data is involved in future phases, we will submit a formal application.

7.5 Risk Management Overview

This project proactively manages the risks associated with AI illusions and workplace dynamics through modular development and technical transparency.

Reference

1. Gadde, Hemanth. (2020). AI-Assisted Decision-Making in Database Normalization and Optimization. International Journal of Machine Learning Research in Cybersecurity and Artificial Intelligence. http://ijmlrcai.com/index.php/Journal/article/download/175/183/312
2. Su, Y., Zhou, J., Ying, J. et al. Computing infrastructure construction and optimization for high-performance computing and artificial intelligence. CCF Trans. HPC 3, 331–343 (2021). https://doi.org/10.1007/s42514-021-00080-x
3. Othman, R. S., & Yasin, H. M. (2025). AI-driven database optimization: Machine learning applications in database management systems. International Journal of Scientific World, 11(2), 62–70. https://doi.org/10.14419/3r6dyh15
4. Oloruntoba, O. (2025). AI-driven autonomous database management: Self-tuning, predictive query optimization, and intelligent indexing in enterprise IT environments. World Journal of Advanced Research and Reviews, 25(2), 1558–1580. https://doi.org/10.30574/wjarr.2025.25.2.0534
5. PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/
6. PostgreSQL Global Development Group. pg_stat_statements Module. https://www.postgresql.org/docs/current/pgstatstatements.html
7. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 2016.
8. Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
9. Chase, H. LangChain Documentation. https://python.langchain.com/
10. ChromaDB Documentation. https://docs.trychroma.com/
11. Turnbull, J. The Prometheus Monitoring System. 2018.
12. Grafana Labs. Grafana Documentation. https://grafana.com/docs/
13. PgBouncer Documentation. https://www.pgbouncer.org/
14. PgBouncer Documentation. https://www.pgbouncer.org/
15. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/
16. Van Aken, D., Pavlo, A., Gordon, G. J., and Zhang, B. Automatic Database Management System Tuning Through Large-scale Machine Learning. SIGMOD, 2017.
17. Marcus, R., Negi, P., Mao, H., et al. Bao: Making Learned Query Optimization Practical. SIGMOD, 2021.
18. Marcus, R., and Papaemmanouil, O. Deep Reinforcement Learning for Join Order Enumeration. aiDM Workshop, SIGMOD, 2018.
19. CIDR Workshop on AI for Databases and Databases for AI. https://ai4db.org/